Generated and started: 13/10/2026

Brief Quest (C# API) - 4-Week Project Plan

Project Overview

Objective: Build and deploy "Brief Quest", a C# / ASP.NET Core API (with a small Next.js front end) that turns an assignment brief into a checklist of quests, helping students, especially those with ADHD and dyslexia, track every requirement

Duration: 4 weeks (20 working days, 6 hours each = 120 hours)

Team: Solo developer, with Claude as tutor and reviewer

Resources: Free tools and free-tier services only

Target: A deployed portfolio project that proves backend skills (ASP.NET Core, PostgreSQL, Docker, CI, testing, authentication) and adds a small React/TypeScript front end

Locked-In Decisions

- **Backend:** C# with ASP.NET Core on .NET 10 LTS
- **Database:** PostgreSQL. Local Docker for development; Neon free tier for the live demo (chosen by Claude on your behalf, easy to change later because only the connection setting differs)
- **Accounts:** simple user accounts from the start, with sign-up, login, per-user data and account deletion. Exact login approach to be confirmed against the .NET 10 docs on Day 11
- **Front end:** Next.js + React + TypeScript as a thin client of three screens (sign in, submit a brief, view and tick off quests). The C# API is the only backend; no business logic in Next.js
- **AI:** Gemini free tier called from the API, with a per-user daily cap stored in PostgreSQL. Bring-your-own-key is a stretch goal. Test only with public or fictional briefs
- **To decide on Day 14:** where the API itself is hosted (options and trade-offs will be presented first)
- **Not in this project:** camera scanning, tutor tools, the RPG quest map (stretch)

How Each Day Works

- 5 hours of planned tasks (about 1-1.5h learning, the rest building) plus 1 hour for the daily review with Claude, an evidence-log entry and buffer
- You write the code; Claude reviews, explains and gives hints before solutions
- Friday: demo to Claude and a short quiz on the week's concepts

- If a day feels too big, do only the first task and stop; that still counts as progress
- Scope tiers: **Must** (API, database, tests, CI, deployment), **Should** (accounts, per-user cap, React screens), **Stretch** (quest map, file upload, read-aloud, own-key setting, end-to-end tests). If you fall behind, cut Stretch first, then Should; never Must

---

Week 1: Walking Skeleton and First AI Call (30h)

13/10/26

Day 1: Setup and first endpoint (5h + review)

- ☐ Learn: ASP.NET Core overview, minimal APIs versus controllers (1h 30m)
- ☐ Install the .NET 10 SDK, an editor and Docker; create the Web API project (1h)
- ☐ Run a hello endpoint and open the generated API docs page (1h)
- ☐ Create a public GitHub repo, `.gitignore`, first commit, README stub (1h)
- ☐ Create a free Google AI Studio key (billing off); keep it in user-secrets, never in the repo (30m)

Done when: the API runs locally, the repo is public and no key is committed.

"Minimal APIs" recommended for new projects. Controllers are used in the MVC patterns.

ASP.NET Core WebApi project.

app.MapGet("/", ()=> "Hello"); and in the .http file, add GET {{localhostURL}}, then run and press the "send request" that pops up above your GET.

In GitHub create a repo without gitignore, public and no readme file. In VSCode terminal do: git new gitignore, git init, git branch -M main, git add . , git commit -m "my message", git remote add origin <the address GitHub shows you>, git push -u origin main.

dotnet user-secrets init, dotnet user-secrets set "Gemini:ApiKey" "get-your-key-from-aistudio". You can check it (in private!!) with "dotnet user-secrets list". Note: this key is now registered in the config file and will only work in Development environment. You can check it's being reached by making an endpoint and run it through the http file, like this:

```
app.MapGet("/config-check", (IConfiguration config) =>
    string.IsNullOrEmpty(config["Gemini:ApiKey"]) ? "Key missing" :
"Key found"
);
```

Day 2: Models, endpoints and dependency injection (5h + review)

- ☐ Learn: records and DTOs, dependency injection in ASP.NET Core (1h 30m)
- ☐ Define Brief, Requirement, Outcome and Criterion models (1h 30m)
- ☐ Build `POST /briefs` and `GET /briefs/{id}` with in-memory storage (1h 30m)
- ☐ Test with an `.http` file or the API docs page (30m)

Done when: you can post a brief and read it back.

Requirement => ExtractedRequirementDTO. I also put the create and update methods in the same file, no need to create a lot of unnecessary files.

Day 3: First Gemini call (5h + review)

- ☐ Learn: calling the Gemini API; check whether the official .NET package is current, else use REST (1h)
- ☐ Build an extraction service that asks Gemini for structured JSON (2h)

```
public async Task<string> SendAsync(object body){

   using var response = await _httpClient.
   PostAsJsonAsync($"v1/models/{_geminiModel}:generateContent", body);
   var text = await response.Content.ReadAsStringAsync();
   .....
}
```

dotnet user-secrets set "ConnectionStrings:BriefQuest" "Host=localhost;Port=5247;Database=briefquest;Username=briefquest;Password=password" --project api

(Day 7)

- ☐ Configure secrets and environment variables properly (30m)
- ☐ Add error handling and retry with backoff for rate-limit (429) responses (1h 30m)

In SendAsync(), run all the code in a for-loop with no max condition
for(int attempt = 1; ; attempt++)
1: send request and get a response from the client. 2:count up attempts. Check response.StatusCode, do you need to retry or is it a genuine error (e.g. 404)? 3: if retry, wait for some random time then try again, if you've done enough tries or it's a genuine error then:
throw new GeminiUnavailableException(statusCode, responseText);

Done when: posting a brief returns a model response, and failures are handled.

Day 4: Validation (5h + review)

- ☐ Learn: validating structured output against typed models (1h)
- ☐ Parse model JSON into typed objects and reject invalid output (1h 30m)
- ☐ Check that every quoted source sentence really exists in the brief (1h 30m)
- ☐ Return validated requirements from the endpoint (1h)

The structure of the Gemini output is JSON, the typed object is C#.

Normalise(briefText).Contains(sentence, StringComparison.Ordinal);

Done when: invalid or invented model output is rejected.

Day 5: Tests and week review (5h + review)

- ☐ Learn: xUnit basics (1h)
- ☐ Write unit tests for validation and the source-sentence check (2h)
- ☐ Write an integration test for `POST /briefs` (1h)
- ☐ Demo to Claude and take the weekly quiz (1h)

Fact or Theory? Test a statement or test a theory about a method with different input?
Arrange => Act => Assert

Done when: tests pass and you can explain the week's concepts aloud.

Week 1 Deliverables:

- Running ASP.NET Core API that turns a pasted brief into validated requirements with source sentences
- Unit and integration tests
- Free AI key set up safely

Week 1 Assessment Criteria:

- ☑ `POST /briefs` returns a validated list of requirements, each with its source sentence
- ☑ Invalid model output is rejected, with a test proving it
- ☑ At least 6 tests pass
- ☑ No API key in the repository
- ☐ You can explain dependency injection in your own words

Explain it to Claude: Why must the API key stay on the server? What does dependency injection give you in this project? How does your code catch an invented source sentence?

Week 2: PostgreSQL and the Quest Engine (30h)

Day 6: SQL fundamentals and schema (5h + review)

- ☐ Learn: free interactive SQL tutorials - SELECT, WHERE, JOIN, GROUP BY (2h)
- ☐ Draft tables and relationships: users, briefs, outcomes, criteria, quests, AI cache, usage (1h 30m)
- ☐ Review the schema with Claude and revise (1h 30m)

Done when: the schema is agreed with Claude and written down.

Count how many lines include each Bloom Level in the CommandWords table, and order them by count in a descending order:
SELECT "BloomLevel", COUNT(*) AS "WordCount" FROM "CommandWords" GROUP BY "BloomLevel" ORDER BY "WordCount" DESC;

NOTE: The DB is case sensitive! When querying, remember to "quote" labels if using "CapitalLetters"

Basically a simple class diagram with relationships (1-1, 0-* etc)

The brief-quest-architecture-and-learning-noted.pdf

Day 7: PostgreSQL in Docker with EF Core (5h + review)

- ☐ Learn: Entity Framework Core with the Npgsql provider (1h)
- ☐ Run PostgreSQL in Docker with a compose file (1h)
- ☐ Create the DbContext and your first migration (2h)
- ☐ Persist briefs to the database instead of memory (1h)

Done when: briefs survive an API restart.

(1) Docker (environment container, like a virtual machine) hosts many frameworks/libs, including (2) PostGreSQL (server+RDBMS). Inside PSQL you can create (3) databases that use SQL to CRUD tables. Docker needs to be running in order for you to access your db.

The compose file is a .yml config file to specify ports and stuff. If the DB is a file cabinet, then DBContext is the clerk that has a decipher notepad (EFCore) to convert between API and DB languages and how it works to connect the two ends. The API department only speaks to the cleark, never goes to the file cabinet directly.
- To make briefs persist: (1) Add them to the db tracking list, (2) save them to the db.

- Migrations! EFCore's main job, to create DB templates and rules from inside the codebase, e.g. by converting a list of entities into a table of entries, with properties/class variables as columns. Every time you have a new/updated entity or property to store in the DB, in the API side, you (1) code it, (2) create a MIGRATION ORDER (dotnet ef migrations add NewStuff), then (3) carry out that order (dotnet ef database update). (You need to install a few frameworks/plugins for it to work first)

Day 8: Quests and relationships (5h + review)

- ☐ Learn: EF Core relationships and loading related data (1h)
- ☐ Save extracted outcomes, criteria and quests (2h)
- ☐ Add GET endpoints with joins and filters (1h 30m)
- ☐ Seed reference data: about 30 command words you write yourself (30m)

Done when: a brief returns its outcomes, criteria and quests from the database.

The extracted data is returned in JSON, so it's easy to create an entity (e.g. new Outcome()) and fill it with info from the JSON object, then store that Outcome object in the db.Outcomes table.

in the C# endpoints, add input parameters that you want to use to filter/find objects with. In the http query, use [URL]?var1=Merit&var2=Todo

Day 9: Progress, tiers and caching (5h + review)

- ☐ Learn: SQL indexes and aggregates; reading a query plan (1h)
- ☐ Add `PATCH` for quest status and progress queries by tier (2h)
- ☐ Add an AI response cache table keyed by a hash of the brief (1h 30m) Moved to Day 13
- ☐ Test the queries and the cache (30m)

Done when: progress by tier is correct and repeat briefs use the cache.

For brief 4, show each tier, how many quests it has, and how many of them are Done:

```
SELECT "Tier",
COUNT(*) AS "Count",
Count(*) FILTER (WHERE "Status"='Done') AS "Done"
FROM "Quests"
WHERE "BriefID"=4
GROUP BY "Tier"
ORDER BY "Tier";
```

The result:

```
Tier           | Count | Done
---------------+-------+------
 Distinction |       1 |    0
 Merit       |       1 |    1
             |       3 |    2
```

The same query in C# code:

```
//1. Get quests  //2. where briefID = id
//3. Create groups of tiers from the quests (You'll get a dictionary
<Tier?, Quest>)   //4. For each group, create a TierProgressDTO
 var progress = await dbcontext.Quests.AsNoTracking()
            .Where(q => q.BriefID == id)
            .GroupBy(q => q.Tier)
            .Select(groupOfTier => new
TierProgressDTO(groupOfTier.Key, groupOfTier.Count(),
groupOfTier.Count(q=> q.Status == QuestStatus.Done)) )
            .ToListAsync();
```

Day 10: Docker, CI and week review (5h + review)

- ☐ Learn: Dockerfile for ASP.NET Core, GitHub Actions basics (1h)
- ☐ Write a Dockerfile for the API (1h 30m)
- ☐ Add a GitHub Actions workflow that builds and runs tests (1h 30m)

☐ Demo to Claude and take the weekly quiz (1h)

Done when: CI is green on every push and the container runs.

Week 2 Deliverables:

- PostgreSQL schema with migrations
- Quest engine with progress queries by tier
- AI response cache
- Dockerfile and automated CI

Week 2 Assessment Criteria:

☐ A full brief can be saved, queried and progressed through quest statuses
☐ Schema has foreign keys and at least one useful index
☐ At least two SQL queries written by you, not generated
☐ Repeat briefs are served from the cache
☐ CI is green and the Docker image builds

Explain it to Claude: Write a JOIN that lists criteria for one outcome. Why index a column? What does a migration solve?

Week 3: Accounts, Security and AI Limits (30h)

Day 11: Accounts, part 1 (5h + review)

☐ Learn: ASP.NET Core Identity and token-based login; confirm the current .NET 10 approach in the docs (1h 30m)
☐ Add Identity with PostgreSQL storage (2h)
☐ Build register and login endpoints (1h 30m)

Done when: you can register, log in and receive a token.

Day 12: Accounts, part 2 (5h + review)

☐ Learn: authorisation basics (30m)
☐ Protect endpoints so users only see their own briefs (2h)
☐ Write tests for the authorisation rules (1h 30m)
☐ Add an account-deletion endpoint that removes the user's data (1h)

Done when: one user cannot read another user's briefs, proven by tests.

Day 13: Per-user AI cap and rate limiting (5h + review)

☐ Learn: ASP.NET Core rate limiting middleware (1h)

- ☐ Store a per-user daily AI cap in PostgreSQL (2h)
- ☐ Return friendly errors when the cap or the free tier is exhausted (1h)
- ☐ Confirm billing is off on the Google AI project (30m)
- ☐ Draft a short privacy notice (30m)

Done when: a user who hits the daily cap gets a clear message, with a test.

Day 14: Quality, API docs and hosting decision (5h + review)

- ☐ Learn: standard problem-details error responses (30m)
- ☐ Add consistent error responses and input validation (1h 30m)
- ☐ Describe every endpoint in the generated API docs (1h)
- ☐ Add structured logging (1h)
- ☐ Choose the API host: Claude presents options, you decide (1h)

Done when: errors are consistent, docs are readable and the host is chosen.

Day 15: Command words, first steps and week review (5h + review)

- ☐ Learn: Bloom's taxonomy as data (30m)
- ☐ Build a command-word decoder endpoint backed by PostgreSQL (1h 30m)
- ☐ Generate a first 2-minute step for each quest using the AI (1h 30m)
- ☐ Demo to Claude and take the weekly quiz (1h 30m)

Done when: the decoder returns plain-language meanings and each quest has a first step.

Week 3 Deliverables:

- Working accounts with per-user data and account deletion
- Per-user daily AI cap
- Consistent errors, documented endpoints and logging
- Command-word decoder and first-step quests
- API hosting decision made

Week 3 Assessment Criteria:

- ☐ Users can register, log in, and only access their own data, with tests
- ☐ Account deletion removes the user's data
- ☐ The daily AI cap is enforced per user and tested
- ☐ Errors are consistent and endpoints are documented
- ☐ Command-word decoder reads from the database

Explain it to Claude: What is the difference between authentication and authorisation? Why store the cap in the database? What personal data do you hold, and how can a user remove it?

Week 4: React Page, Deploy and Show (30h)

Day 16: Next.js basics and sign-in (5h + review)

- ☐ Learn: react.dev core concepts and Next.js app router basics (2h)
- ☐ Create the Next.js + TypeScript app and call the API; configure CORS on the API (1h 30m)
- ☐ Build the sign-in and register screen (1h 30m)

Done when: a user can sign in from the web page.

Day 17: Submit a brief and see quests (5h + review)

- ☐ Learn: fetching data and managing state in React (1h)
- ☐ Build the submit-brief screen (1h 30m)
- ☐ Build the quest list with highlighted source sentences (2h 30m)

Done when: a pasted brief appears as a list of quests with sources.

Day 18: Progress and polish (5h + review)

- ☐ Tick off quests and show progress by tier (2h)
- ☐ Add loading and error states; check keyboard use and contrast (1h 30m)
- ☐ Deploy the front end (Vercel Hobby, personal use) (1h 30m)

Done when: the front end is live and ticks persist through the API.

Day 19: Deploy the API and write the docs (5h + review)

- ☐ Deploy the API to the chosen host, connected to Neon (2h)
- ☐ Smoke-test the whole flow and fix CORS or configuration problems (1h)
- ☐ Write the README: architecture diagram, decisions, limitations (1h 30m)
- ☐ Record a 90-second demo video (30m)

Done when: a stranger can open the live demo and follow the README.

Day 20: Validate and career (5h + review)

- ☐ Have 3-5 students or tutors try it; record feedback (2h)
- ☐ Fix the top three issues (1h 30m)
- ☐ Write up the project for your CV and LinkedIn; pin the repo; update your CV tag to .NET 10 (1h)
- ☐ Plan Weeks 5-6 (30m)

Done when: feedback is gathered, top issues are fixed and the project is on your CV.

Week 4 Deliverables:

- Live Next.js front end and live C# API
- README, architecture diagram and demo video
- User feedback and CV write-up

Week 4 Assessment Criteria:

- ☐ The three screens work end to end against the live API
- ☐ Both services are deployed and the README explains how they fit together
- ☐ Feedback from at least 3 real users is recorded
- ☐ CI is green on the final commit
- ☐ You can demo and explain every part without notes

Explain it to Claude: What does CORS protect against? Why keep business logic in the API and not in Next.js? What would you improve first with another month?

Final Project Assessment (rate yourself on Day 20)

Aim for at least Merit in every row, and Distinction in two.

Area	Pass	Merit	Distinction
Backend function	Core flow works end to end	Plus accounts and per-user cap	Plus stretch features
Code quality	Readable C#, sensible structure	Dependency injection, clear layers	Could be handed to another developer
Data and SQL	Schema with relationships	Indexes, migrations, own queries	Explains query plans and trade-offs
Security and privacy	Passwords never stored in plain text	Authorisation tested, account deletion	Privacy notice and data retention thought through

Area	Pass	Merit	Distinction
Testing and CI	Unit tests and CI	Plus integration tests	Tests catch real bugs you introduced
Communication	Clear README	Architecture diagram, demo video	You explain decisions confidently

---

Tools & Resources

Development Stack

- **Backend:** C# on .NET 10 LTS, ASP.NET Core Web API
- **Data:** PostgreSQL, Entity Framework Core with Npgsql
- **Auth:** ASP.NET Core Identity (approach to confirm on Day 11)
- **AI:** Gemini free tier via Google AI Studio; official .NET package status to verify, REST as fallback
- **Testing:** xUnit, integration tests
- **DevOps:** Docker, GitHub Actions
- **Front end:** Next.js, React, TypeScript
- **Hosting:** Neon (database), Vercel Hobby (front end, personal use only), API host to be chosen on Day 14

Free Learning Sources

- Microsoft Learn: ASP.NET Core and C# documentation
- Official docs for EF Core, Npgsql, xUnit, Docker and GitHub Actions
- react.dev and the Next.js official learn course
- Free interactive SQL tutorials (SQLBolt, SQLZoo)

Suggested Folder Structure

```
brief-quest/
├── api/
│   ├── Endpoints/        (brief, quest, account endpoints)
│   ├── Services/         (Gemini extraction, validation,
cap)
│   ├── Data/             (DbContext, migrations, seed data)
│   └── Dockerfile
├── api.tests/            (unit and integration tests)
├── web/                  (Next.js front end)
├── docker-compose.yml    (local PostgreSQL)
└── .github/workflows/    (CI)
```

Risk Management

Project-Specific Risks

- **Two stacks at once:** C# API plus React/TypeScript adds load
- **Overwhelm and burnout:** 30-hour weeks while learning new tools
- **Accounts add data-protection duty:** you will store emails and password hashes
- **Free-tier changes:** AI quotas, hosting and database limits can change without notice
- **Cold starts:** free hosts may be slow on first visit; Neon wakes automatically (reported at about half a second), while free web hosts such as Render's can take about a minute
- **AI inaccuracy:** the model may misread or invent requirements

Mitigation Strategies

- Keep the front end to three screens; the API is the priority
- Do only the first task on a hard day; schedule lighter days; cut Stretch first
- Collect the minimum personal data, hash passwords through Identity, add account deletion and a privacy notice; seek proper advice before any real-world launch
- Check official pricing pages before each phase; keep billing off
- Say in the README that the first load may be slow
- Verify every requirement against its exact source sentence and reject anything else

Budget Considerations

- **Hosting, database, AI, CI:** $0 on free tiers
- **Domain (optional):** around $10-15 a year
- **Total estimated cost:** $0 (optional $10-15)
- Vercel Hobby is for personal, non-commercial use only; hosting terms would need to change if you monetise

Success Metrics

Technical Metrics

- **Tests:** unit and integration tests passing in CI on every push
- **Security:** users can only reach their own data, proven by tests
- **Reliability:** friendly handling when the AI quota or daily cap is reached
- **Documentation:** every endpoint described in the API docs

Launch Metrics

- **Real users:** 3-5 students or tutors try it and give feedback
- **Portfolio:** public repo, live demo, README and 90-second video
- **Career:** project written up on your CV and LinkedIn, tagged with .NET 10

Post-Launch Roadmap

Weeks 5-6: Deepen and Extend

- PostgreSQL practice: query tuning and more advanced SQL
- Azure basics or another cloud platform, since many listings mention cloud experience
- End-to-end tests

Later

- The RPG quest map, file upload, read-aloud and bring-your-own-key
- Tutor-side brief clarity checker
- Camera scanning for photographed briefs

Daily Checklist:

- ☐ Run build and tests before committing
- ☐ Commit with meaningful messages
- ☐ Write one entry in your evidence log (what you built, what you learned)
- ☐ Review the day with Claude: paste code or a summary
- ☐ Check no secrets are in the repo

☐ Update progress against the weekly criteria